

Task 1:

Task Name

Create an Animal class and override the sound() method in child classes.

Objective:

1. To understand the concept of method overriding in Java.
2. To demonstrate runtime polymorphism using inheritance.
3. To implement method overriding using parent and child classes.
4. To learn how subclasses provide their own implementation of a parent class method.

Problem Analysis:

In Java, method overriding occurs when a subclass provides a specific implementation of a method that is already defined in its parent class. This is an example of runtime polymorphism. In this task, a parent class named Animal is created with a method called sound(). Two child classes Dog and Cat extend the Animal class and override the same method. Each subclass prints its own sound message. This demonstrates how the same method behaves differently depending on the object.

Input:

No input is required for this program.

Process:

A parent class Animal is created with a method called sound(). Then two subclasses Dog and Cat extend the Animal class and override the sound() method with their own implementation. Objects of each class are created in the main method and the sound() method is called to display different outputs.

Output:

The program prints different sounds for different animals.

Algorithm

1. Start the program.
2. Create a class named Animal.
3. Create a method sound() inside the Animal class.

4. Create a class Dog that extends Animal.
5. Override the sound() method in the Dog class.
6. Create a class Cat that extends Animal.
7. Override the sound() method in the Cat class.
8. Create the main method.
9. Create objects of Animal, Dog, and Cat classes.
10. Call the sound() method using the objects.
11. Display the output.
12. End the program.

Source Code:

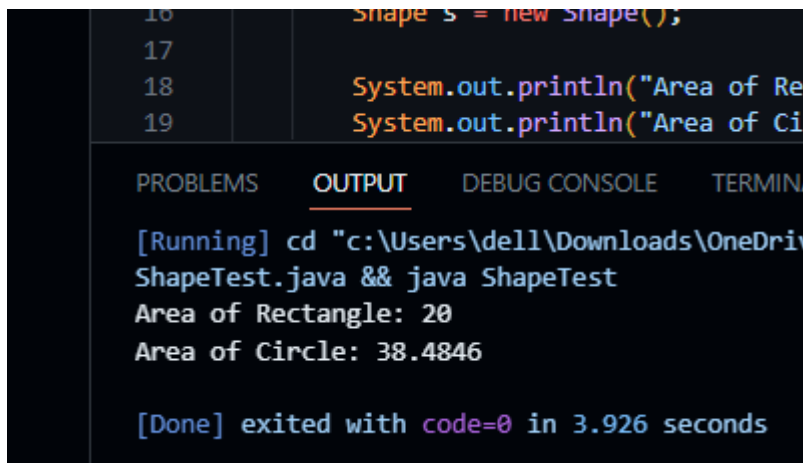
```
class Animal {  
    void sound() {  
        System.out.println("Some generic sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Cat extends Animal {  
    void sound() {  
        System.out.println("Cat meows");  
    }  
}  
  
public class TestAnimal {
```

```

public static void main(String[] args) {
    Animal a = new Animal();
    Dog d = new Dog();
    Cat c = new Cat();
    a.sound();
    d.sound();
    c.sound();
}
}

```

Output:



```

16      Shape s = new Shape();
17
18      System.out.println("Area of Re
19      System.out.println("Area of Ci

```

PROBLEMS	OUTPUT	DEBUG CONSOLE	TERMINAL
[Running] cd "c:\Users\dell\Downloads\OneDrive" && java ShapeTest Area of Rectangle: 20 Area of Circle: 38.4846 [Done] exited with code=0 in 3.926 seconds			

Discussion

This program demonstrates method overriding in Java. The parent class `Animal` contains a method called `sound()`, and the subclasses `Dog` and `Cat` override this method with their own implementations. When the method is called using different objects, different outputs are produced. This shows how Java supports runtime polymorphism, where the method execution depends on the object type.

Task 2

Task Name

Create a Shape class and override the draw() method in child classes.

Objective:

1. To understand method overriding using inheritance.
2. To demonstrate runtime polymorphism in Java.
3. To implement a parent class and multiple subclasses.
4. To observe how different classes override the same method.

Problem Analysis:

Method overriding allows a subclass to redefine a method of its parent class. This helps in implementing runtime polymorphism. In this task, a parent class named Shape contains a method draw(). Two subclasses Circle and Rectangle extend the Shape class and override the same method. Each subclass prints a different message to represent drawing a specific shape.

Input:

No input is required for this program.

Process:

A parent class Shape is created with a method draw(). Two subclasses Circle and Rectangle extend the Shape class and override the draw() method. In the main method, objects of Shape, Circle, and Rectangle are created and their draw() methods are called.

Output:

The program prints messages for drawing different shapes.

Algorithm:

1. Start the program.
2. Create a class named Shape.
3. Create a method draw() in the Shape class.
4. Create a subclass Circle that extends Shape.

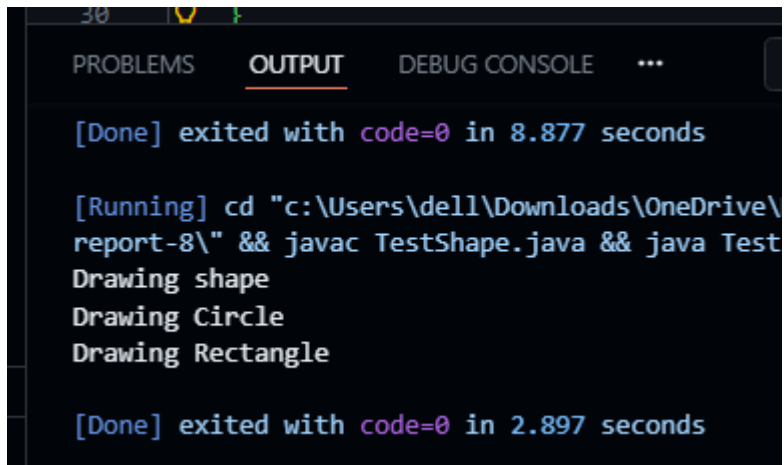
5. Override the draw() method in the Circle class.
6. Create another subclass Rectangle that extends Shape.
7. Override the draw() method in the Rectangle class.
8. Create the main method.
9. Create objects of Shape, Circle, and Rectangle.
10. Call the draw() method using the objects.
11. Display the output.
12. End the program.

Source Code:

```
class Shape {  
    void draw() {  
        System.out.println("Drawing shape");  
    }  
}  
  
class Circle extends Shape {  
    void draw() {  
        System.out.println("Drawing Circle");  
    }  
}  
  
class Rectangle extends Shape {  
    void draw() {  
        System.out.println("Drawing Rectangle");  
    }  
}  
  
public class TestShape {  
    public static void main(String[] args) {
```

```
Shape s = new Shape();  
Circle c = new Circle();  
Rectangle r = new Rectangle();  
s.draw();  
c.draw();  
r.draw();  
}  
}
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  ...  
[Done] exited with code=0 in 8.877 seconds  
  
[Running] cd "c:\Users\dell\Downloads\OneDrive\  
report-8\" && javac TestShape.java && java Test  
Drawing shape  
Drawing Circle  
Drawing Rectangle  
  
[Done] exited with code=0 in 2.897 seconds
```

Discussion:

This program demonstrates runtime polymorphism through method overriding. The parent class Shape defines a general method draw(), while the subclasses Circle and Rectangle override the method with their own implementations. When the method is called using different objects, Java executes the appropriate method based on the object type. This shows how method overriding enables flexible and dynamic behavior in object-oriented programming.